Nibin Nihad — Portfolio Website: Project Report

> A complete guide to how this website is built, written so you can explain every part of it confidently in an interview.
>
> **Live site:** https://nibin-nihad-portfolio.vercel.app **Report date:** 12 November 2026

---

1. Project summary (your 30-second answer)

> "It's my personal portfolio, built with **Next.js 15, React 19 and TypeScript**, styled with **Tailwind CSS**, animated with **Framer Motion**, and deployed on **Vercel**. The content (projects, experience, skills, education) lives in **one typed data file**, so I can update the site without touching the components. Its signature feature is a **'swarm rebuild' intro**: when the page opens, a team of animated cursors rebuilds the whole page piece by piece on an HTML canvas, drawing boxes, typing text and sweeping in images, as the visitor scrolls. It's fully **responsive**, works on phones, respects **reduced-motion** settings, and builds as a **static page** for fast loading."

Item	Detail
Type	Single-page personal portfolio (static site)
Framework	Next.js 15 (App Router)
Language	TypeScript (strict mode)
UI library	React 19
Styling	Tailwind CSS 3 + a small global CSS file
Animation	Framer Motion 12, plus hand-written `requestAnimationFrame` loops and Canvas 2D
Hosting	Vercel (production URL above)
Code size	~4,300 lines across 23 source files
Rendering	Statically prerendered at build time (`○ Static` in the build output)
First load	~180 kB of JavaScript for the home page

---

2. Technologies and tools

2.1 Core stack

Technology	Version	What it does in this project
Next.js	15.5	React framework. Uses the **App Router** (`src/app/`), automatic code-splitting, image handling (`next/image`), font optimisation (`next/font`) and static prerendering.
React	19	Builds the UI out of components. Uses hooks: `useState`, `useEffect`, `useRef`, `useCallback`.
TypeScript	5.9	Adds types. `strict: true` is on, so mistakes like a missing field or a wrong type fail the build.
Tailwind CSS	3.4	Utility-first CSS: styles are written as class names (`px-4 py-16 sm:py-24`). Also handles responsive breakpoints (`sm:`, `lg:`) and custom brand colours.
Framer Motion	12	Declarative animations: fade/slide-ins when a section scrolls into view (`whileInView`), the hero entrance, scroll-linked animation (`useScroll`) in Projects, and the 3D tilt card.
Lucide React	0.475	The icon set (GitHub, LinkedIn, mail, map pin, arrows, etc.). Icons are tree-shaken, so only the ones used are shipped.
canvas-confetti	1.9	The confetti burst when the mini puzzle game is solved.

2.2 Build and quality tools

Tool	Purpose
ESLint 9 + `eslint-config-next`	Linting with the `next/core-web-vitals` and `next/typescript` rule sets. They catch React-hooks mistakes, accessibility issues and performance anti-patterns.
PostCSS + Autoprefixer	Processes Tailwind and adds browser prefixes automatically.
Node.js	The development runtime (v24 on the dev machine).
npm	Package manager (`package.json` / `package-lock.json`). `npm audit` reports **0 vulnerabilities**.
Vercel CLI (via `npx vercel`)	Deploys the site to production.
Google Fonts via `next/font`	**Inter** for body and headings, **JetBrains Mono** for code-style labels. Self-hosted at build time, so there's no layout shift and no request to Google from the visitor's browser.

2.3 Browser APIs used directly (good interview talking points)

API	Where	Why
Canvas 2D	Swarm rebuild	Draws all cursors, selection boxes, rulers and guides in **one** canvas, at 60 fps.
requestAnimationFrame	Swarm, profile ribbon, skills carousel	Smooth, frame-synced animation loops that pause when not needed.
IntersectionObserver	Profile ribbon, Replay button	Runs work only when an element is on screen.
ResizeObserver	Swarm engine, profile ribbon	Re-measures layout when content or window size changes.
CSS `clip-path`	Swarm	Reveals real page elements progressively (typing text, sweeping images) without copying them.
CSS 3D transforms (`perspective` , `translate3d` , `rotateY`)	Profile ribbon, skills carousel, tilt card	Real 3D motion without WebGL.
Range API (`document.createRange`)	Swarm blueprint	Measures each rendered line of text so the "typing" caret follows the real line breaks.
Clipboard API	Contact	"Copy email" button.
`matchMedia`	Ribbon, puzzle, pre-paint script	Detects screen size and `prefers-reduced-motion` .
sessionStorage	Swarm	Records that the rebuild has played this session.

3. Folder structure

```
Potfolio.nibin/
├── public/                    Static files served from the site root
│   ├── nibin-hero.webp        Hero cut-out portrait
│   ├── resume.pdf             Downloaded by the "Resume" button
│   └── suii-logo.png          Small blue logo above the "T" of PORTFOLIO
├── src/
│   ├── app/                   Next.js App Router
│   │   ├── layout.tsx         <html>/<body>, fonts, SEO metadata, swarm pre-paint script
│   │   ├── page.tsx           The home page: puts all sections in order
│   │   ├── globals.css        Base colours, scrollbar, swarm CSS rules, reduced-motion rules
│   │   └── icon.svg           Favicon
│   ├── components/            One file per section / widget
│   │   ├── Navbar.tsx         Top bar, mobile menu, Resume button
│   │   ├── ScrollProgress.tsx  Thin progress bar at the very top
│   │   ├── Hero.tsx           Hero area: stage + bento cards (bio, "Currently", featured work,
stats)
│   │   ├── HeroStage.tsx      Editorial hero: "PORTFOLIO" watermark, portrait, name
│   │   ├── About.tsx          Profile section + 3D card ribbon
│   │   ├── Skills.tsx         3D curved skills carousel
│   │   ├── Experience.tsx     Internship timeline
│   │   ├── Projects.tsx       Stacking project panels
│   │   ├── Education.tsx      Degree and school cards
│   │   ├── Certifications.tsx  Certifications + personal skills
│   │   ├── Contact.tsx        Contact details, message form, mini game
│   │   ├── PuzzleGame.tsx     3x3 swap puzzle with confetti
│   │   ├── Footer.tsx         Footer + back-to-top
│   │   ├── SectionHeader.tsx  Shared section title component
│   │   ├── TiltCard.tsx       Reusable card that tilts toward the pointer
│   │   └── swarm/             The swarm rebuild feature (see section 6)
│   │       ├── SwarmRebuild.tsx  React wrapper: canvas, Escape-to-skip, Replay button
│   │       ├── engine.ts         Animation engine (cursors, scheduling, drawing)
│   │       ├── blueprint.ts      Scans the page DOM into a build plan
│   │       └── constants.ts      On/off switch and shared attribute names
│   └── data/
│       └── portfolio.ts       ALL site content, fully typed
├── package.json               Dependencies and scripts
├── tailwind.config.ts         Brand colours and fonts
├── tsconfig.json              TypeScript settings (strict, "@/" path alias)
├── next.config.ts             Next.js settings (React strict mode)
├── .eslintrc.json             Lint rules
├── .vercelignore              Files Vercel should not upload
└── PROJECT_REPORT.md          This report
```

4. Architecture: how the site is put together

4.1 Rendering flow

1. `layout.tsx` creates the HTML shell, loads the two fonts, sets SEO metadata, and injects a tiny inline **pre-paint script**.
2. `page.tsx` renders the sections in order: **Hero** → **About** → **Skills** → **Experience** → **Projects** → **Education** → **Certifications** → **Contact** → **Footer**.
3. At build time Next.js **prerenders the page to static HTML** (`next build` reports `○ / Static`), so the first view arrives fast and search engines can read all the text.
4. In the browser, React **hydrates** the HTML (attaches interactivity), and the client components (`"use client"`) start their animations.

4.2 Content is separated from layout

All text lives in `src/data/portfolio.ts` as one typed object, `portfolioData`:

Key	Contents
`personal`	name, title, location, email, phone, LinkedIn, GitHub, `resumeUrl`, status, `shortBio`, `aboutNarrative`
`navigation`	menu items and their `#section` links
`experiences`	Bairuha Tech (Full Stack Developer Intern), CodeMe Hub (Data Analyst Intern)
`projects`	FinTrack Pro, NSS Connect, Bio-Cyber Hybrid Neuromorphic Chemical Sensor
`skills`	8 categories (Languages, Frontend, Backend, Mobile, Databases, Data Analysis, Tools & DevOps, CS Fundamentals)
`education`	B.Tech CSE (AWH Engineering College), 12th, 10th
`certifications`	UAS / Drone boot camp (NIELIT Calicut)
`personalSkills`	Problem Solving, Analytical Thinking, Continuous Learning, Communication and Teamwork

TypeScript **interfaces** (`Project`, `Experience`, `SkillCategory`, `EducationItem`, `CertificationItem`) describe each shape, so a typo or missing field is caught at build time.

Interview line: *"I separated content from presentation. To add a project I only edit the data file; the Projects component maps over the array."*

4.3 Server vs client components

- Components without `"use client"` render on the server (at build time) and ship no extra JavaScript.
- Interactive ones (`Navbar`, `About`, `Skills`, `Contact`, `PuzzleGame`, `HeroStage`, the swarm) are marked `"use client"`.

- The **swarm engine is lazy-loaded** with a dynamic `import("./engine")`, so its ~1,400 lines don't slow down the first page load.

4.4 Design system

- **Theme:** black, white and **deep cobalt** `#2C3480` on a warm light-grey page (`#f3f3f1`).
- Tailwind is extended with a `cobalt` palette (`DEFAULT`, `deep`, `light`, `bright`, `soft`).
- Fonts are exposed as CSS variables (`--font-inter`, `--font-mono`) and used through Tailwind's `font-sans` / `font-mono`.
- `SectionHeader` gives every section the same "// LABEL + big headline + cobalt accent" style, so the design stays consistent.

---

5. Section-by-section walkthrough

Navbar (`Navbar.tsx`)

- Fixed top bar with section links, a **Resume** button (downloads `public/resume.pdf`) and **Get in touch**.
- Below the `xl` breakpoint it collapses into a **hamburger menu**. The menu button uses `aria-expanded` for screen readers.
- `ScrollProgress.tsx` draws a thin cobalt bar at the very top that fills as you scroll.

Hero (`Hero.tsx` + `HeroStage.tsx`)

- **Editorial "stage"** sitting straight on the page (no card): a faint guide grid with "+" marks, a giant faded **"PORTFOLIO"** watermark (with the small blue logo standing on the "T"), the cut-out portrait on the right, and **NIBIN NIHAD.** set very large at the bottom left.
- Entrance animations use Framer Motion. With reduced motion turned on, they become instant.
- Below the stage, a **bento grid** of cards: bio + social links, a cobalt **"Currently"** card (Full Stack Developer Intern @ Bairuha Tech), **Featured Work** (the 3 projects with tech tags), and stats (3 projects, 2 internships, 2026 B.Tech CSE, First Class).

Profile / About (`About.tsx`)

- Lead paragraph where the **words light up one after another** as you scroll (each word is a `motion.span` animating its opacity).
- **3D card ribbon** (desktop ≥1024px, motion allowed): scroll position drives a progress value `p` from 0 to 1.
  - Two huge headlines ("CODE QUALITY" / "USER IMPACT.") slide past each other like trains.
  - The three profile cards ride a **3D rail** (a curve computed in code, resampled by arc length so the spacing stays even), then fly into a 3-column grid.
  - `p` is **smoothed** (it eases toward the scroll target), so the motion feels fluid rather than jerky.
  - The ribbon plays **every time** you scroll down through it and rewinds when you scroll up. A short "hold" keeps the finished grid pinned briefly at the end.

- Phones and reduced-motion users get a plain, stacked grid instead.

Skills (`Skills.tsx`)

- **Curved 3D carousel**: cards in the middle are flat, and cards towards the edges bend around a virtual cylinder (`rotateY` + `translateZ`) and fade.
- Drifts slowly by itself. Can be **dragged/swiped**, scrolled sideways with a trackpad or Shift+wheel, or moved with the **arrow keys**.
- Loops seamlessly (positions wrap with a modulo).
- Has `role="region"` / `aria-roledescription="carousel"`, and a caption announced through `aria-live`.

Experience (`Experience.tsx`)

- Cards for each internship: role, company, dates, a badge, key responsibilities, pages delivered (e.g. Podcast, About HLB, Publications) and the tech stack.

Projects (`Projects.tsx`)

- **Stacking panels**: each project pins under the navbar (`position: sticky`) while the next slides over it. Panels underneath shrink and dim (Framer Motion `useScroll`) so the stack reads as depth.
- Each panel shows the category, title, description, tech tags, "What it does" and "How I built it".

Education and Certifications

- B.Tech card (2022–2026, CGPA 6.69, First Class), 12th (78.25%) and 10th (84.44%).
- Certifications card plus personal-skills tiles.

Contact (`Contact.tsx` + `PuzzleGame.tsx`)

- Email (with **copy-to-clipboard**), phone/WhatsApp, location, LinkedIn and GitHub.
- **Message form:** on submit it opens the visitor's email app with the name, subject and message pre-filled (a `mailto:` link). There's no server or database, which keeps the site free to host and private.
- **Mini game:** a 3×3 **swap puzzle**. Tap two tiles to swap them; when every tile is home, a hidden message reads in full and **confetti** fires. Each tile is a full-size copy of a real-DOM "picture", shifted so only its ninth shows, which keeps the text sharp at any size.

Footer (`Footer.tsx`)

- Name, copyright, "Thank you for scrolling" and a **Back to top** button.

---

6. Signature feature: the Swarm Rebuild

6.1 What the visitor sees

1. The page opens **blank**; a swarm of colourful cursors **orbits the site icon** in the middle of the screen ("scanning").
2. The cursors fan out and **rebuild the page**, Figma/design-tool style:
   - **Boxes:** crosshair drag with a blue `W × H` size badge and a grey placeholder, then the real fill fades in.
   - **Text:** a text box is dragged out, then the text is **typed** with a blinking caret.
   - **Images:** grey placeholder, then the image **sweeps in** from top to bottom.
   - **Repeated items** (cards, tags): the first is built, the rest are **duplicated** with red spacing measurements.
   - **Rulers and red guides** slide in for sections whose text lines up on shared left edges.
3. Work unlocks **as you scroll**: anything you scroll past is finished instantly, so you're never blocked.
4. At the end the cursors **flock to your mouse pointer** and fade out. A small **Replay** button appears in the corner.
5. **Escape** skips it at any time.

6.2 How it works (technical)

Step	File	Explanation
1. Pre-paint gate	`constants.ts` → `layout.tsx`	An inline script in `<head>` runs **before the first paint** and adds `data-swarm-pending` to `<html>`. CSS hides `<main>` so there's no flash of the finished page. It is skipped when the visitor prefers reduced motion or opened a `#section` link. A **4-second CSS failsafe** reveals the page if the JavaScript ever fails.
2. Lazy load	`SwarmRebuild.tsx`	The React component mounts a full-screen `<canvas>` and **dynamically imports** the engine, so it isn't in the main bundle.
3. Blueprint scan	`blueprint.ts`	Walks the DOM of every `<section>` and the footer and classifies each element as a **box**, **text**, **media**, **line**, **duplicate** or **fade** task, with its page rectangle and parent (children wait until their parent box is mostly built). It also finds **guide lines** (shared left edges of text). Elements can override this with `data-swarm-as="fade
4. Hide real content	`engine.ts`	Each task's element gets `data-swarm="hidden"` (opacity 0). The **real DOM is revealed** later with `clip-path`, never copied, so the result is exactly the real page.
5. Scheduler	`engine.ts`	Each frame, tasks that are on screen and "ready" are offered to idle cursors (nearest first). The crowd **grows with the amount of work and scroll speed** and shrinks when idle. Everything runs from **one** `requestAnimationFrame` **loop**, without React re-rendering every frame.

Step	File	Explanation
6. Drawing	`engine.ts`	All cursors, boxes, handles, badges, rulers and guides are drawn on **one Canvas 2D**, scaled for retina screens (`devicePixelRatio`). The rulers are cached in offscreen canvases.
7. Finish	`engine.ts`	Restores every element's original styles, removes the attributes and listeners, and tells React to show the Replay button.

Special case: the 3D profile ribbon. Cards moving in 3D can't be measured reliably, so while the rebuild runs the cards ride the ribbon as **blank panels** (`data-swarm-defer`). When they land in the grid, `About.tsx` fires a custom event (`swarm:append`). The engine then scans the cards, adds them to the build, and **re-measures them every frame**, because they sit on a pinned (sticky) stage whose page position changes while you scroll.

Live widgets (the Skills carousel) hold still while the swarm is running (`data-swarm-running` on `<html>`), so they can be built in place. Framer Motion's own entrance animations are made instant during the build (`MotionGlobalConfig.instantAnimations`) so the two animations don't fight.

Credit (be honest about this in interviews): the visual idea is inspired by Joel Califa's *Swarm* website and the open-source *Swarm Rebuild* Chrome extension. This implementation (engine, scheduler, DOM blueprint, ribbon integration) was written for this portfolio. No code was copied, because that repository has no licence.

Main switch: `SWARM_ENABLED` in `src/components/swarm/constants.ts` turns the whole feature on or off.

---

7. Responsive design (mobile)

- **Mobile-first Tailwind breakpoints:** base styles target phones; `sm:` (640px), `lg:` (1024px) and `xl:` (1280px) add the larger layouts.
- Tested at **360px** and **390px** phone widths and at **1440px** desktop: **no horizontal scrolling** anywhere.
- Phone-specific choices:
  - Hamburger navigation; bento cards stack into one column.
  - Hero: status pills sit below the "PORTFOLIO" watermark, the portrait is smaller, and the title has a soft backdrop so it stays readable over the photo.
  - Section padding is reduced on phones (`py-16` instead of `py-24`) to cut down empty scrolling.
  - The 3D ribbon becomes a simple stacked grid.
  - Small links ("View all", contact links, "Hire me") have **enlarged tap areas** (extra padding offset by a negative margin, so nothing moves).
  - The Replay button becomes a compact round icon.
- Fluid type with CSS `clamp()`, e.g. the name is `clamp(56px, 10.5vw, 150px)`, so headings scale smoothly between phone and desktop.

---

8. Accessibility

- **Semantic HTML:** `<nav>`, `<main>`, `<section id=…>`, `<article>`, `<footer>`, real `<button>`s and links, and one `<h1>` (your name).
- **Reduced motion:** a global CSS rule shortens all animations, the swarm doesn't auto-play, and the ribbon falls back to a grid when `prefers-reduced-motion: reduce` is set.
- **Keyboard:** Escape skips the swarm; the skills carousel works with the arrow keys; focus rings are visible (`focus-visible:outline`); tabbing ahead during the swarm instantly builds everything up to the focused element.
- **Screen readers:** `aria-label` on icon-only buttons, `aria-hidden` on decorative layers (watermark, grid, logo), `aria-expanded` on the menu button, an `aria-live` caption on the carousel, and a screen-reader announcement while the rebuild plays.
- **Descriptive alt text** on the portrait.

---

9. Performance

- **Static prerendering:** the HTML is ready at build time and served from Vercel's CDN.
- **~180 kB first-load JavaScript** for the home page (shared framework chunks ~103 kB).
- **Code-splitting:** the swarm engine loads only when needed (dynamic `import()`).
- `next/font`**:** fonts are self-hosted, with `display: swap` and no layout shift.
- **WebP image** for the hero portrait, with `priority` so it loads first.
- **Animation efficiency:**
  - A single canvas and a single rAF loop for the swarm instead of hundreds of DOM nodes.
  - Ruler graphics are cached offscreen.
  - Animation loops **stop when idle or off-screen** (IntersectionObserver, `p ≡ target` check).
  - `will-change: transform` on moving cards; transforms instead of layout properties.

---

10. SEO and metadata

Defined in `layout.tsx` through the Next.js `Metadata` API:

- **Title:** "Nibin Nihad | Software Developer"
- **Description** and **keywords** (Software Developer, React, Next.js, Python, Kerala, India...)
- `authors`, `creator`, and **Open Graph** tags (title, description, site name) for link previews on LinkedIn, WhatsApp and similar apps.
- `lang="en"` on `<html>`, and a favicon (`app/icon.svg`).
- All text is in the prerendered HTML, so search engines can index it.

---

11. Deployment (Vercel)

Item	Value
Platform	Vercel
Account / project	`nibinnihad` / `nibin-nihad-portfolio`
Production URL	https://nibin-nihad-portfolio.vercel.app
Build command	`next build` (detected automatically)
Link file	`.vercel/project.json` connects this folder to the Vercel project
Excluded from upload	`.vercelignore` (node_modules, .next, .vercel, caches)

Deploy a new version:

```
npx vercel --prod --yes
```

Interview line: *"Vercel builds the Next.js app, prerenders the page, and serves it from a global CDN with HTTPS. Redeploying is one command."*

12. Commands you should know

```
npm install          # install dependencies
npm run dev          # start the dev server at http://localhost:3000 (hot reload)
npm run build        # production build (also runs type-checking and lint)
npm run start        # serve the production build locally
npm run lint         # run ESLint
npx tsc --noEmit     # type-check only
npx vercel --prod    # deploy to production
```

13. How to update the site

I want to...	Do this
Change my bio, email, phone or links	Edit `personal` in `src/data/portfolio.ts`
Add a project / internship / skill	Add an item to `projects` / `experiences` / `skills` in the same file
Replace my resume	Put the new PDF in `public/` named `resume.pdf`
Change the brand colour	`cobalt` in `tailwind.config.ts` (and `--accent` in `globals.css`)

I want to...	Do this
Turn the swarm animation off	Set `SWARM_ENABLED = false` in `src/components/swarm/constants.ts`
Change the build speed of the swarm	`BUILD_SPEED` in `src/components/swarm/engine.ts`
Publish the changes	`npx vercel --prod --yes`

---

14. Quality checks performed

- ✅ `next build` passes: type-checking and lint with **0 errors / 0 warnings**.
- ✅ Browser checks on the production build at **1440px desktop** and **390px phone**: no console errors or warnings, no failed network requests, no broken images, nothing left hidden after the swarm finishes.
- ✅ All in-page menu links point to sections that exist; the resume PDF and images load.
- ✅ `npm audit`: **0 vulnerabilities**.
- ✅ Unused code, images and scripts were removed in a cleanup (dead components, unused npm packages `clsx` and `tailwind-merge`).

Known limitations (good to admit honestly)

- **No automated tests** (unit/E2E). Checks were done manually and with scripted headless-browser runs. A natural next step is Playwright tests.
- The **contact form uses** `mailto:`, so it depends on the visitor having an email app. A next step is a small serverless API route or a form service.
- The **Open Graph URL** in `layout.tsx` is set to `https://nibinnihad.dev`, not the live Vercel URL. It should be updated if that domain isn't owned.
- The swarm animation is a **creative showcase**. It respects reduced motion and is skippable, but it adds JavaScript compared with a plain static page.

---

15. Likely interview questions and suggested answers

Q: Why Next.js instead of plain React (Create React App / Vite)? A: Next.js gives me static prerendering for fast first load and SEO, built-in image and font optimisation, file-based routing, and zero-config deployment on Vercel. For a portfolio that recruiters open once, first-load speed and indexable content matter.

Q: What is the App Router? A: Next.js's newer routing system in the `app/` folder. `layout.tsx` wraps every page and `page.tsx` is the route's content. Components are server components by default, and I opt into client-side interactivity with `"use client"` only where I need state or effects.

Q: Why TypeScript? A: The content file is a big nested object. With interfaces, a missing field or a typo fails the build instead of breaking the live site. Strict mode also catches null/undefined bugs early.

Q: Why Tailwind CSS? A: Fast to iterate, consistent spacing and colours from one config, responsive variants inline (`sm:`, `lg:`), and unused classes are removed from the production CSS, so the stylesheet stays small.

Q: How does the swarm animation avoid hurting performance? A: The engine is lazy-loaded, everything is drawn on a single canvas from one `requestAnimationFrame` loop, React doesn't re-render per frame, ruler graphics are cached offscreen, and the real page is revealed with `clip-path` rather than duplicated. It's skipped for reduced-motion users and can be stopped with Escape.

Q: How did you make sure the page never stays hidden if something breaks? A: Three safety nets: the engine is wrapped in try/catch and reveals everything on any error, a CSS animation reveals `<main>` after 4 seconds if the engine never starts, and the pre-paint script only hides content when JavaScript is actually running.

Q: How does the 3D ribbon work without WebGL? A: I compute a 3D curve in code, resample it by arc length so cards stay evenly spaced, and for each card set a CSS `translate3d` + `rotateY` inside a parent with `perspective`. Scroll progress drives the position along the curve, smoothed with an easing factor each frame.

Q: How is the site responsive? A: Mobile-first Tailwind classes with breakpoints, fluid type with `clamp()`, layouts that collapse to one column, heavy 3D effects swapped for simple grids on small screens, and enlarged tap targets. I verified it at 360px and 390px with no horizontal overflow.

Q: What would you improve next? A: Add Playwright end-to-end tests, replace the `mailto:` form with a serverless function, add analytics, fix the Open Graph URL and add an OG image, and run a Lighthouse audit to tune image sizes.

Q: What was the hardest bug? A: The profile cards ride a pinned (sticky) 3D stage, so their position on the page keeps changing while you scroll. The rebuild measured them once, so it built them off-screen or drew its boxes in the wrong place. I fixed it by handing the cards to the swarm the moment they land, adding a short pinned "hold" at the end of the ribbon, and re-measuring those elements every frame.

---

16. Projects featured on the site (quick reference)

Project	Type	Stack
FinTrack Pro	Personal finance dashboard (frontend)	React.js, TypeScript, Recharts, Tailwind CSS
NSS Connect	Volunteer management website (full stack)	Python, Django, HTML5, CSS3, SQLite, JavaScript

Project	Type	Stack
Bio-Cyber Hybrid Neuromorphic Chemical Sensor	IoT + mobile + ML gas sensor	Flutter, Dart, Python, ESP32, MQ gas sensors, Brian2 SNN

Experience	Role
Bairuha Tech (Present)	Full Stack Developer Intern: production pages for the official HLB HAMT website (Next.js, React, TypeScript, Tailwind, Docker, Git)
CodeMe Hub International (Jun–Jul 2025)	Data Analyst Intern: Python (Pandas, NumPy), Excel, Power BI, Matplotlib

Good luck with the interview!